DOCENTE	Paolo BELLONI [paolo.belloni@its-ictpiemonte.it]
Unità Formativa	**Sistemi di Virtualizzazione**
Argomento	La virtualizzazione a livello SO – I container

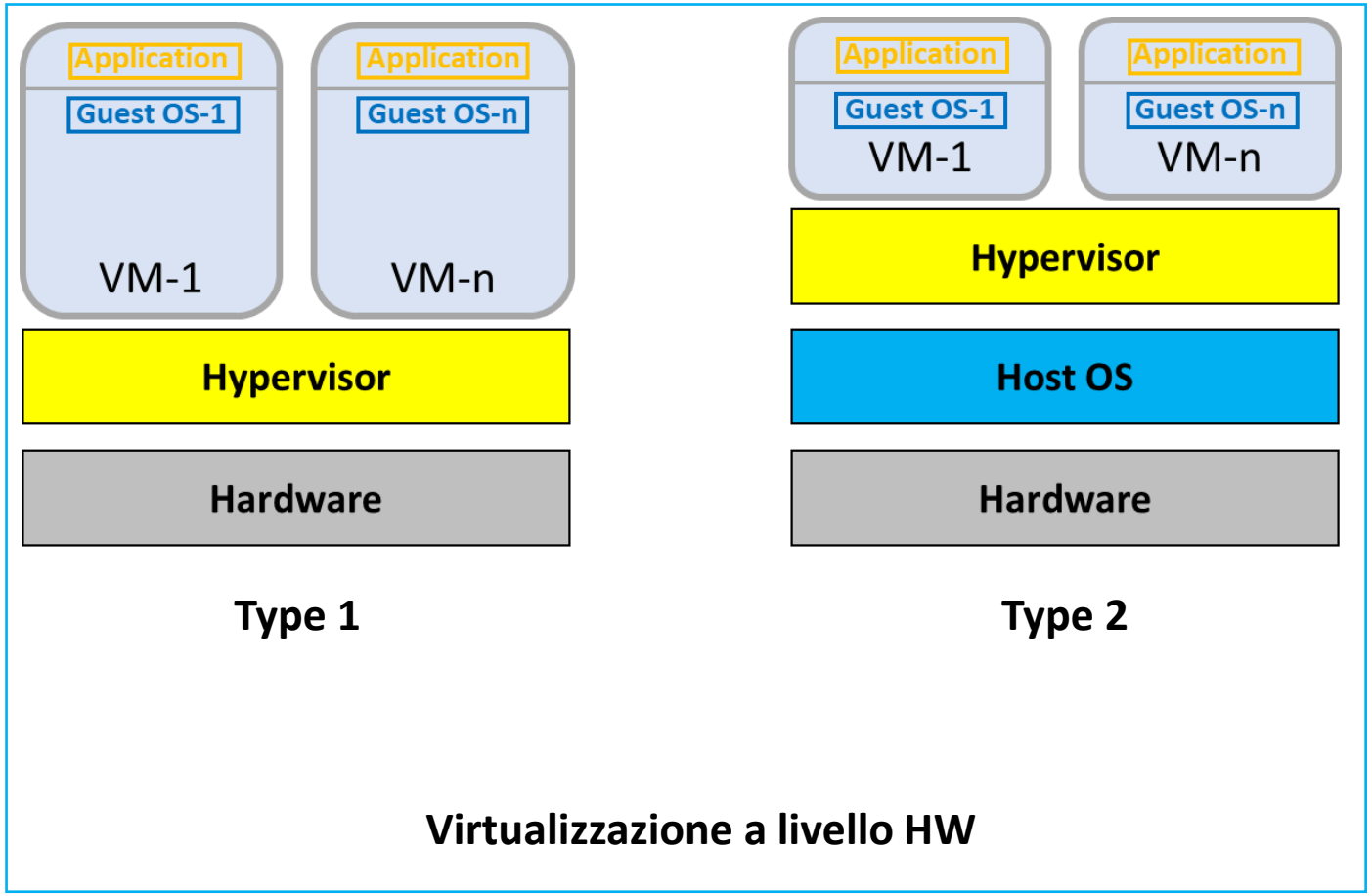
Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

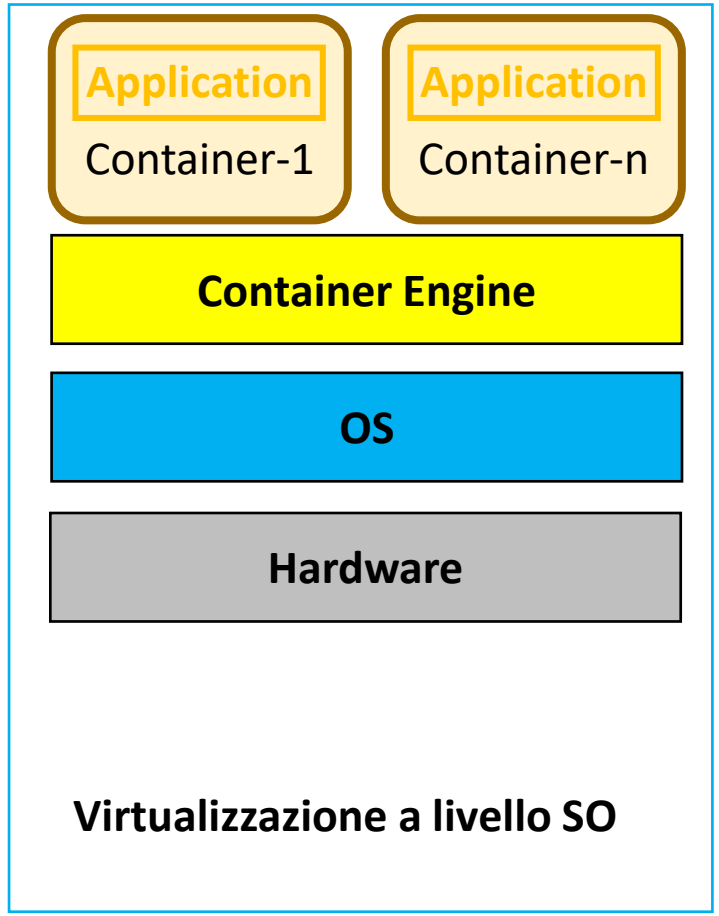


I container

- Nasce negli anni Cinquanta rivoluzionando il commercio mondiale.
- L'idea del Container è attribuita a Malcom McLean, un imprenditore americano nel campo dei trasporti, che, nel 1956, si rese conto che caricare l'intero "corpo del camion" con dentro la merce sulla nave sarebbe stato più semplice ed efficace che stivarne il carico.
- La containerizzazione ha sviluppato enormemente i commerci mondiali favorendo un impulso notevole nel campo del trasporto marittimo e negli scambi tra America, Europa e Asia. La diffusione dei container ha favorito la crescita del trasporto via mare e lo sviluppo dell'intermodalità, consentendo ai vettori marittimi di soddisfare e anticipare i bisogni emergenti del settore.

La Virtualizzazione a livello di SO 2/2





Heavy-weight

Light-weight

Virtualizzazione a livello di SO: meccanismi Unix

- I sistemi operativi Unix e Unix-like (Linux) offrono dei meccanismi, integrati nativamente nel kernel del SO, per la costruzione dei container, ovvero per virtualizzare ed isolare i processi.
  - **chroot** (disponibile già dal 1979);
  - **cgroups** e **namespaces** (Linux, dal 2002).
- *chroot* (change root directory)
  - Permette di cambiare la root folder che il processo corrente in esecuzione e i sui derivati vedono.
- *Namespaces*
  - Permette di isolare quello che un insieme di processi può vedere ed accedere nell'ambiente operativo (processi, porte, file, …).
- *cgroups* (control groups)
  - Permette di limitare, misurare ed isolare l'uso di risorse (CPU, memoria, rete, blocchi di I/O) di un determinato insieme di processi.

Linux Namespace

- Mount namespace: fornisce una root.
- Pid (Process ID) namespace: i processi vedono solo se stessi e i proprio derivati (children).
- Network namespace: permette al container di avere il proprio network stack dedicato.
- User namespace: permette ad un utente non-root su un host di essere mappato su un utente root all'interno del container.
- UTS (Unix Time-Sharing): fornisce un hostname dedicato.
- IPC (Inter-Process Communication): permette di creare spazi di memoria non sovrapposti tra processi che condividono una memoria fisica.

Uso di comando *chroot*

Approfondimento

Scopo: creare un mini-jail equipaggiato con i comandi bash e ls.

- J=$HOME/jail # creazione jail e le relative directory
- mkdir -p $J
  mkdir -p $J/{bin,lib64,lib}
  cd $J
- cp -v /bin/{bash,ls} $J/bin # copiare /bin/bash e /bin/ls nel jail
- ldd /bin/bash # identificazione delle dependencies di bash

  linux-vdso.so.1 => (0x00007fff8d987000) libtinfo.so.5 => **/lib64/libtinfo.so.5** (0x00000032f7a00000) libdl.so.2 => **/lib64/libdl.so.2** (0x00000032f6e00000) libc.so.6 => **/lib64/libc.so.6** (0x00000032f7200000) **/lib64/ld-linux-x86-64.so.2** (0x00000032f6a00000)
- cp -v /lib64/libtinfo.so.5 /lib64/libdl.so.2 /lib64/libc.so.6 /lib64/ld-linux-x86-64.so.2 $J/lib64/ # copiare le librerie necessarie a bash nel jail
- ldd /bin/ls # identificazione delle dependencies di ls
- cp -v xxx # copiare le librerie necessarie a ls nel jail
- sudo chroot $J /bin/bash # entrare nel jail
- ls /
  ls /etc/
  ls /var/ # usare alcuni comandi base

Prodotti per la creazione dei container

Gli sviluppatori possono creare dei container utilizzando i comandi nativi che il kernel dei SO mette a disposizione; tuttavia esistono sul mercato delle piattaforme che rendono più facile, semplice e sicuro creare, implementare e gestire i container.

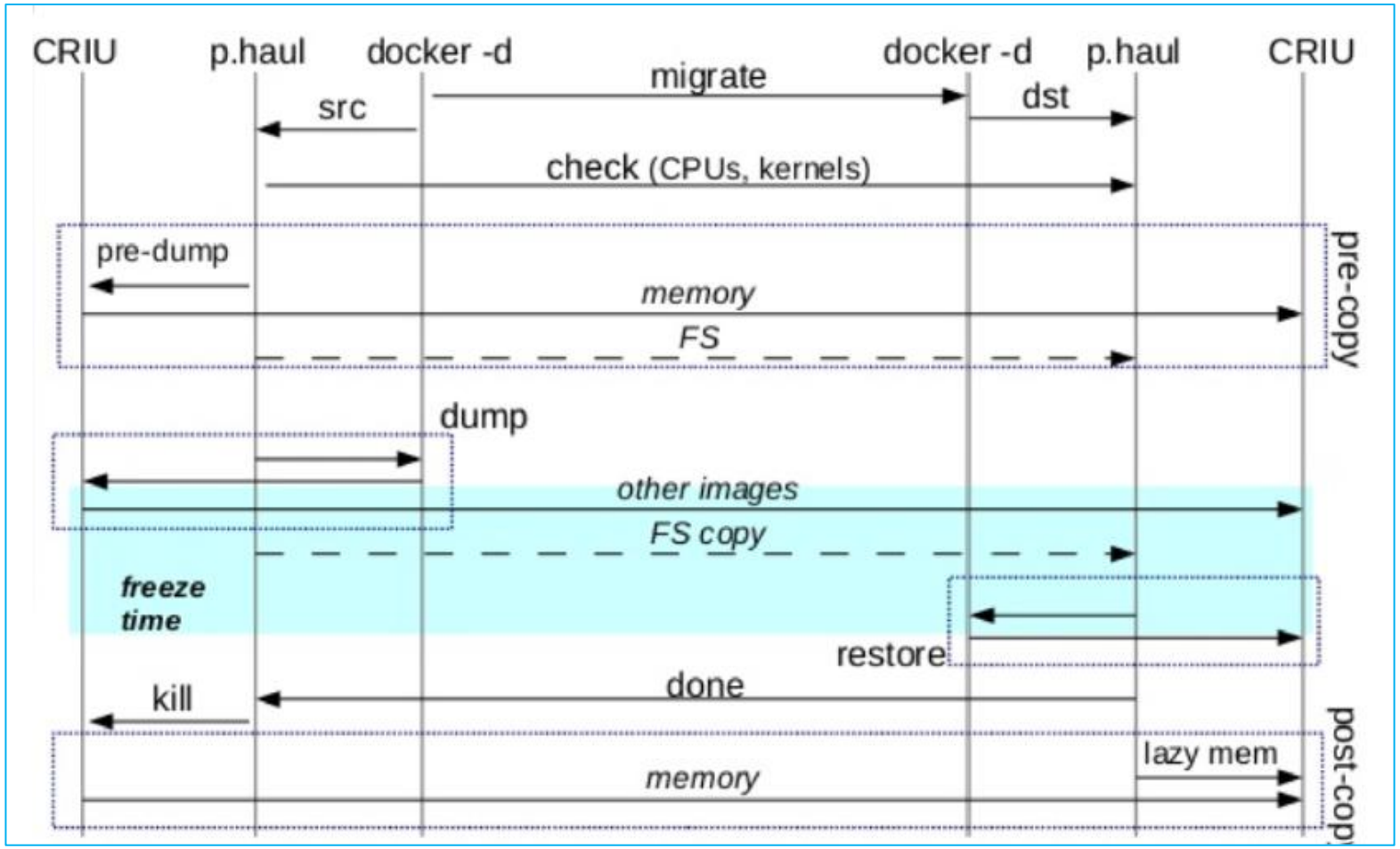
- **IBM LPAR** (sviluppato per la prima volta nel **1976**, consente di creare delle partizioni logiche (Logical Partition), ovvero un sottinsieme di risorse hardware virtualizzato come un computer separato.
- **FreeBSD Jail** (**2000**, consentiva di ripartire il sistema in più sottosistemi isolati chiamati jail).
- **Solaris Zones**/Containers (**2004**, Oracle Solaris).
- **OpenVZ** (Open Virtuozzo, for Linux, **2005**).
- **LXC** (LinuX Containers, **2008**; tra i componenti fondamentali del progetto vi è lo sviluppo dei cgroups, creati da Google nel 2006, i quali consentono di controllare e limitare l'utilizzo delle risorse da parte di un processo o di un gruppo di processi).
- **Docker** (**2013**, il sistema di container Linux più utilizzato in campo IT. Docker è un progetto open source sviluppato dalla compagnia Dotcloud (in seguito rinominata Docker) sulla base di LXC, che, attraverso ulteriori avanzamenti, diventò ciò che è la tecnologia Docker di oggi. Dal 2014 Docker non utilizza più LXC come ambiente di esecuzione di default, sostituito dalla propria libreria libcontainer scritta nel linguaggio di programmazione Go. Docker è un ecosistema complesso ma molto intuitivo per il deployment e la gestione dei container, ricco di funzionalità tra le quali un sistema di immagini, registri locali e globali per i container e un'interfaccia a linea di comando).
- **Rkt** (**2014**, sviluppato da Red Hat, che ha donato il progetto alla Cloud Native Computing Foundation).

Operazioni sui container

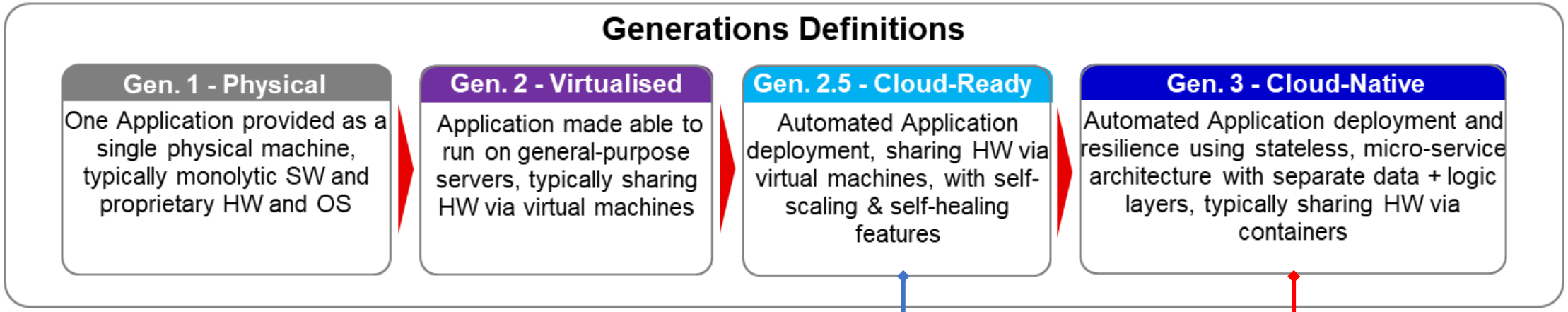
Come per le VM, anche i container possono essere ridimensionati e migrati.

- Il ridimensionamento dinamico dei container, in termini di CPU, di memoria e di I/O, consiste nel modificare i limiti imposti al container al momento della sua creazione.
  - Può avvenire senza restart del container.
  - Si adotta una soluzione di basso livello: il limite del *cgroups* può essere modificato in corsa.
- I container possono essere migrati. In questo caso, serve:
  - Salvarne lo stato;
  - Trasferire lo stato;
  - Ripristinare lo stato.
- Il salvataggio, il trasferimento ed il ripristino dello stato di un container avvengono solo con i suoi task congelati (si parla di *migration downtime*).
- La migrazione dei container risulta più complicata di quella di VM.
  - Non è direttamente supportata, per cui serve utilizzare funzionalità addizionali fornite da strumenti specifici.
  - Funzionalità **CRIU** (Checkpoint and Restore In User Space), fa il dump e restore.
  - Tool **P.Haul** (Process HAUler), che, oltre a CRIU, si utilizza per eseguire un pre-check, memory pre-copy e post-copy e per migrare i file system.

Migrazione di Container: un esempio



Tassonomia delle Applicazioni



- Esiste un modo per classificare le applicazioni in base alla soluzione di virtualizzazione adottata ed al livello di automazione.
- Idealmente, si traccia un percorso evolutivo, al cui apice si pone una soluzione nativamente basata sul cloud, sviluppata su container e completamente automatica.

Gen. 2.5 V1 – No Orchestrator
Gen. 2.5 V2 – Orchestrator Authorized
Gen. 2.5 V3 – Orchestrator Driven
V1: basic automation
V2: advanced automation
V3: full automation

Gen. 3 V1 – With VM
Gen. 3 V2 – Different options possible:
Opt1 = VM on containers w/ same app vendor
Opt2 = VM on containers w/o same app vendor
Opt3 = bare metal container

